



2014

Esercizio 2

$T = (V_T, E_T)$ non orientato, connesso, aciclico (albero)

$r \in V_T$

(T, r) albero radicato $V_T = \{0, 1, \dots, n-1\}$

$d[i]$ distanza tra i e r

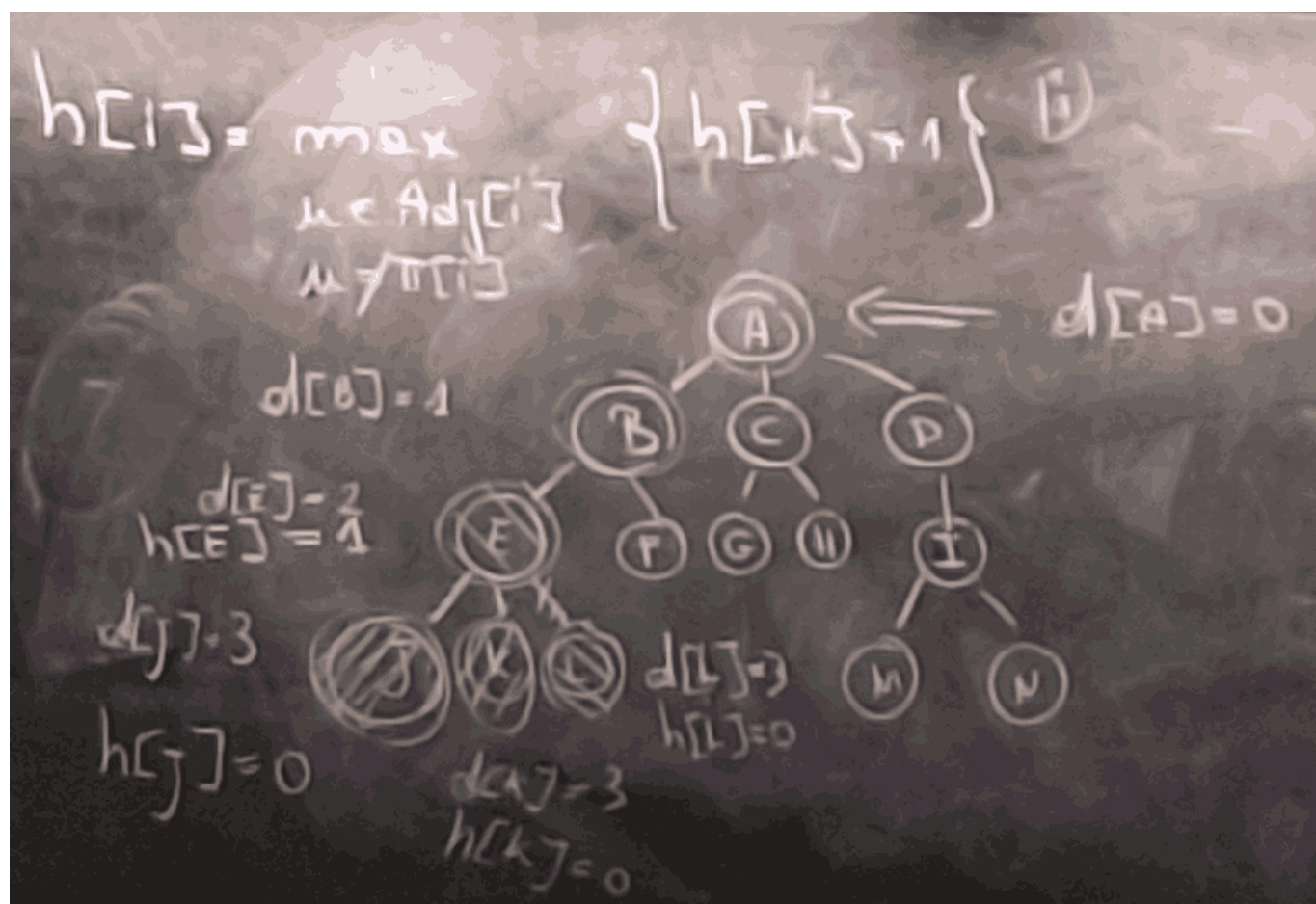
$h[j]$ altezza del sottoalbero di (T, z) radicato in j

Modifichiamo DFS

T albero

$\forall u, v \in T \quad \exists!$ cammino semplice tra u e v

come calcolare $d[j]$?



```
DFS_mod( $T=(V_t, E_t), r$ ):
  for (each  $v$  in  $V_t$ ) {
    color[v] = BIANCO
```

Copia



```

    pi[v] = +∞
    h[v] = 0
}
d[ti] = 0
DFS_visit_mod(T, ri)

```

```

DFS_visit_mod(T, u){
    color[u] = GRIGIO
    for (each v in Adj[u]){
        if (color[v] == BIANCO){
            pi[v] = u
            d[v] = d[u]+1
            DFS_visit_mod(T, v)
            h[u] = max(h[u], h[v]+1) // aggiorno l'altezza del nodo u
        }
    }
    color[u] = NERO
}

```

Copia

Complissità

$\theta(|V|)$

Correttezza

$\forall u, v \in T \quad \exists! \text{ cammino semplice tra } u \text{ e } v$

questa affermazione è fondamentale per la correttezza.

dim) $d[u] = \Sigma(r, u)$ come in BFS

Esiste un unico istante durante DFS_mod in cui $d[u]$ passa da $+\infty$ a $d[u] = \Sigma(r, u)$

Se $h[u] \neq 0 \rightarrow h[u]$ passa da 0 all'altezza dell'albero radicato in u

Per ind. sull'altezza dell'albero radicato in u

Quando DFS_visit_mod(T, v) termina, tutto il sottoalbero radicato in v è già stato esplorato.

Perchè DFS è ricorsiva e G è aciclico.



2012 - compitino

Esercizio 9

- a) BFS ...
- b) SI ...



16/09/2013

Esercizio 2

Sia $G = (V, E)$ un un grafo orientato.

a. Si proponga un algoritmo efficiente per determinare se G è ciclico e se ne determini la complessità.

b. Si disegni un algoritmo che, nell'ipotesi G sia ciclico, determini $G_1 = (V_1, E_1)$ e $G_2 = (V_2, E_2)$ aciclici con le seguenti proprietà:

- $V_1 = V_2 = V$;
- $E_1 \cap E_2 = \emptyset$;
- $E_1 \cup E_2 = E$.

c. Si dimostri la correttezza e si valuti la complessità dell'algoritmo di cui al punto precedente.

```
DFS_visit(G,u){
    .
    .
    .
    for (each v in Adj[u]){
        if (color[v] == BIANCO){
            .
            .
            .
        }
        if(color[v] == GRIGIO){
            print("ciclico")
        }
        .
        .
        .
    }
}
```

[Copia](#)

DFS tutti i Back Edge vengono messi in E_2 (tolta E_1) in E_1 devono andare:

- gli archi di visita (BIANCO)
- gli archi forward e gli archi cross (NERI)

$$\forall (i, j) \in E \begin{cases} \text{se } i < j & (i, j) \in E_1 \\ \text{se } i > j & (i, j) \in E_2 \end{cases}$$



Se E_1 fosse ciclico, sarebbe assurdo



13-06-2013

Esercizio 4

Calcolare lo spazio utilizzato dalla rappresentazione dei grafi tramite liste di adiacenza nei seguenti casi:

- a. il numero di archi uscenti da ogni nodo p una costante;
- b. il grafo è un albero;
- c. il grafo è completo.

Esercizio 7

Sia $G = (V, E)$ un grafo orientato, non pesato. Dati un nodo $s \in V$ e un intero non negativo $k \in \mathbb{N}$, sia L la lista dei nodi a distanza k da s (distanza = lunghezza del cammino minimo).

- a. Si scriva lo pseudo-codice di un algoritmo per determinare L . Si calcoli la complessità e si dimostri la correttezza della soluzione proposta.
- b. Dire se la seguente affermazione è vera o falsa: se la lista L restituita al punto a. non contiene $i \in V$, allora non esiste nessun cammino da s a i di lunghezza k .
- c. Si supponga di avere a disposizione la lista I dei nodi a distanza $k - 1$ da s . è possibile sfruttare I per ottenere una procedura più efficiente di quella proposta al punto a.? Motivare la risposta.

a) Procedura per calcolare L :

```
BFS(G,s){  
  // Se  $g[k] = k$  lo inserisco in  $L$   
  // mi fermo quando in  $Q$  c'è un nodo con distanza  $k + 1$   
}
```

[Copia](#)

complessità: $O(|V| + |E|)$

se $i \notin L$, allora non esiste cammino da s a i di lunghezza k , vero o falso?

$$L_K = u \mid \exists v \in L_{K-1} \text{ and } (v, u) \in E \text{ and } \forall h < k \quad h \notin L_h$$



15-06-2009

Esercizio 6

Sia G un grafo orientato. Si consideri il problema di determinare un nodo v di G tale che v raggiunge il maggior numero possibile di nodi di G .

- a. Descrivere tramite pseudocodice una procedura per risolvere il problema proposto.*
- b. Determinarne la complessità e dimostrarne la correttezza.*

a) calcolo F^* quelli con liste di adiacenza più lunghe in G^* sono quelli che raggiungono più nodi in G

